

Step 4 — Production Wrap — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Nudges, not answers — and this step is honestly closer to "go redo Project 5" than to anything new. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — This is Project 5, not a new design](#)
- [Hint 2 — What's actually different here](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — This is Project 5, not a new design

Basic Version

The README says it plainly: "literally repeat Project 5's Steps 1-4, on this codebase instead of ContentForge's. No new ideas here." Go open Project 5's own hints and solutions (`project_5_contentforge_pro_production/hints_and_solutions/`) and use them as your working reference for this entire step — that's not cheating, it's the point.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Work through Project 5's 4 steps in order, one FastAPI route at a time, against CodeGuard's Supervisor instead of ContentForge's graph. The request/response shape changes (a diff in, a compiled review out, instead of a topic in, a report out) — everything else about *how* you build the route, the database table, the Dockerfile, and the test suite carries over almost unchanged.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

One genuine difference worth thinking about before you copy Project 5's Step 3 rate-limiting code over unchanged: CodeGuard's Supervisor sometimes runs 1 reviewer, sometimes 3, depending on what changed in the diff (Step 3's routing). Does that change what a fair rate limit should look like, compared to ContentForge, where every request did roughly the same fixed amount of work? Worth a real answer, not just a copy-paste.

Difference between Basic, Intermediate, and Advanced: Basic tells you exactly where to look. Intermediate tells you what stays the same and what's genuinely different (the request/response shape). Advanced asks you to notice one place where copying Project 5's logic unchanged might actually be the wrong call, given something real about how this system's cost varies.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — What's actually different here

Basic Version

Your eval suite (Project 5 Step 4's equivalent here) needs its own fixed set of example diffs with known-correct findings — not ContentForge's topics. Write those first; nothing else in this step means anything without them.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Think about what "known-correct findings" means for a diff review, specifically — unlike ContentForge's content briefs, a code review has a much more checkable right answer: does this planted-secret diff get flagged as critical? Does this docs-only diff correctly skip the Security reviewer entirely (proving Step 3's routing, not just the reviewers themselves)? Design your eval tasks to test the routing logic, not just each reviewer in isolation.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Project 5 Step 3's Dockerfile needed no special secrets beyond `OPENAI_API_KEY`. This project's Setup section mentions a style-guide document the Style reviewer's RAG pipeline depends on — think about where that file needs to live relative to the container, and whether it belongs baked into the image (it's not a secret, so `docker history` isn't the concern here) or mounted in at run time the way a real style guide that changes over time probably should be.

Difference between Basic, Intermediate, and Advanced: Basic says to build the eval suite first, same as Project 5. Intermediate pushes you to design eval tasks that actually test Step 3's routing decision, not just each reviewer alone. Advanced asks you to think through one detail Project 5 never had to consider — a non-secret file (the style guide) your RAG pipeline genuinely depends on at runtime.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)